

Software Requirements Specification (SRS)

E-Commerce System with Mock Payment Integration

Prepared By:

Philiswa Maziya 202303961

Zaminkosi Mamba 202303523

Bhekumuzi Gamedze 202303350

Mcondisi Dlamini 202301340

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to describe the requirements for the development of an E-Commerce System with Mock Payment Integration. The system will enable users to browse products, manage a shopping cart, and complete purchases using a simulated (mock) payment gateway.

This document is prepared according to the IEEE Std 830-1998 standard and is intended for developers, testers, project supervisors, and stakeholders.

1.2 Scope

The E-Commerce System is a web-based application that allows customers to shop online and complete transactions using mock payment processing.

The system will:

- Provide product browsing and search functionality
- Allow users to manage shopping carts and wishlists
- Support user registration and authentication

- Handle checkout process with mock payment integration
- Enable order management and history viewing
- Provide basic administrative functions for product and order oversight

The system aims to simulate a realistic e-commerce experience while using mock payment to avoid real financial transactions, making it ideal for demonstration, testing, and academic purposes.

1.3 Definitions, Acronyms, and Abbreviations

- SRS – Software Requirements Specification
- UNESWA – University of Eswatini
- GUI – Graphical User Interface
- SKU – Stock Keeping Unit
- Mock Payment – Simulated payment gateway for testing purposes

1.4 References

- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications
- Software Engineering Course Notes
- Best practices for e-commerce web applications

1.5 Overview

This document describes the functional and non-functional requirements of the E-Commerce System with Mock Payment Integration. It includes system features, user requirements, performance requirements, constraints, and design considerations.

2. Overall Description

2.1 Product Perspective

The system is an independent web-based e-commerce platform designed to provide a complete online shopping experience. It includes user management, product catalog, shopping cart, checkout with mock payment, and basic administration modules.

2.2 Product Functions

The main functions of the system are:

- User registration and authentication
- Product catalog browsing and search
- Shopping cart and wishlist management
- Checkout and mock payment processing
- Order placement and history tracking
- Admin product and order management
- Display of order confirmations

2.3 User Characteristics

The intended users are:

- Customers: General users with basic computer literacy
- Administrators: Store managers with elevated privileges

Users are expected to:

- Have basic internet navigation skills
- Be able to register, browse, and make purchases online

No advanced technical skills are required.

2.4 Constraints

The system operates under the following constraints:

- Must use only mock payment (no real payment gateway)
- Development must be completed within the academic timeline
- Limited development resources
- Responsive design for desktop and mobile

2.5 Assumptions and Dependencies

The system assumes that:

- Users provide accurate information during registration and checkout
- Internet access is available
- Mock payment responses are consistent for testing

The system depends on:

- Web browser compatibility
- Database and backend framework availability
- File/image upload support for products

3. Specific Requirements

3.1 Functional Requirements

3.1.1 User Authentication

- The system shall allow users to register with email, password, and personal details.
- The system shall allow users to log in and log out securely.
- The system shall authenticate users before granting access to protected features.

3.1.2 Product Catalog

- The system shall display products with images, descriptions, prices, and categories.

- The system shall support search and filtering by category, price, and keywords.

3.1.3 Shopping Cart

- The system shall allow users to add, update, and remove items from the cart.
- The system shall calculate totals dynamically.

3.1.4 Checkout and Mock Payment

- The system shall guide users through the checkout process.
- The system shall integrate a mock payment gateway (card details simulation).
- The system shall validate mock payment inputs and return success/failure status.

3.1.5 Order Management

- The system shall generate unique order IDs upon successful payment.
- The system shall allow users to view their order history.
- The system shall allow admins to view and update order status.

3.1.6 Administration

- The system shall allow admins to add, edit, and delete products.
- The system shall provide basic order overview for administrators.

3.2 External Interface Requirements

3.2.1 User Interface

The system interface shall:

- Be simple, modern, and user-friendly
- Provide product grids, cart views, and checkout forms
- Display mock payment form and order confirmation

- Support responsive design for mobile and desktop

3.2.2 Software Interface

The system shall interact with:

- Backend framework (e.g., Node.js, Django, etc.)
- Database system
- Mock payment simulation module

3.2.3 Hardware Interface

The system shall operate on:

- Computers, laptops, and smartphones

3.3 Performance Requirements

- The system shall load product pages within 2 seconds.
- The system shall process mock payment simulation within 3 seconds.
- The system shall support multiple concurrent users.
- Search and cart operations shall be fast and responsive.

3.4 Logical Database Requirements

The database shall store:

- User details (ID, email, password hash, address, role)
- Product information (SKU, name, description, price, stock, images)
- Shopping cart and order records
- Payment transaction logs (mock)

4. Non-Functional Requirements

4.1 Reliability

The system shall provide accurate mock payment processing and order generation with minimal errors.

4.2 Availability

The system shall be available for demonstration and testing purposes.

4.3 Security

- The system shall protect user credentials using hashing.
- The system shall prevent unauthorized access.
- Mock payment data shall not involve real financial information.

4.4 Maintainability

The system shall be developed with modular code for easy updates and maintenance.

4.5 Portability

The system shall operate on different web browsers and operating systems.

5. System Models and Diagrams

5.1 Use Case Diagram

Main actors: Customer, Administrator

Main use cases:

- Register/Login
- Browse/Search Products
- Manage Cart & Wishlist

- Checkout with Mock Payment
- View Orders
- Admin: Manage Products & Orders

5.2 Flowchart

System process flow:

1. User registers/logs in
2. User browses and adds products to cart
3. User proceeds to checkout
4. Mock payment is processed
5. Order is confirmed and saved

6. Assumptions and Constraints

Assumptions

- Users have basic knowledge of online shopping.
- Product data and mock payment test cases are correctly provided.

Constraints

- Limited project development time.
- Use of mock payment only.
- Dependence on internet access for web deployment.

7. Appendices

Appendix A: Tools and Technologies

- HTML, CSS, JavaScript
- Backend framework (Python/Django, Node.js, etc.)

- Database (MySQL / PostgreSQL / MongoDB)
- GitHub for version control

Appendix B: Future Improvements

- Integration with real payment gateways
- Recommendation engine
- Advanced analytics and reporting
- Mobile application development

Conclusion

The E-Commerce System with Mock Payment Integration will provide a complete and realistic online shopping platform suitable for demonstration and academic purposes. The system is expected to showcase essential e-commerce functionalities while maintaining simplicity and reliability.